

# LAB 8

Task # 01 class

Main {

```
String name = "muneeb"; private String  
password = ".communi"; protected String email  
= "muneeb@gmail.com"; public int age = 20;
```

```
public static void main(String[] args) {  
Main main = new Main();
```

```
// Default access: Can be accessed within the same package  
System.out.println("Default Value: " + main.name);
```

```
// Public access: Accessible from anywhere  
System.out.println("Public Value: " + main.age);
```

```
// Private access: Cannot access directly from another class  
System.out.println("Private Value: " + main.password); // Error
```

```
// Protected access: Can be accessed in a subclass or within the same package  
System.out.println("Protected Value: " + main.email);  
}
```

```
}
```

---

public class Main {

```
public static void main(String[] args) {  
TestAccessModifier myClass = new TestAccessModifier();
```

```
// Accessing public member
```

```
System.out.println("Public Variable: " + myClass.publicVar);  
myClass.publicMethod();
```

```
// Accessing private member: This will result in an error
```

```
// System.out.println("Private Variable: " + myClass.privateVar); // Error: privateVar has private  
access
```

```
// myClass.privateMethod(); // Error: privateMethod() has private access
```

```
// Accessing protected member: This is allowed because TestAccessModifiers is in the same  
package
```

## LAB 8

```
System.out.println("Protected Variable: " + myClass.protectedVar);  
myClass.protectedMethod();
```

```
// Accessing default (package-private) member: This is allowed because TestAccessModifiers is in  
the same package
```

```
System.out.println("Default Variable: " + myClass.defaultVar);  
myClass.defaultMethod();
```

```
// Accessing private method through a public method  
myClass.accessPrivateMethod(); // Allowed because accessPrivateMethod() is public  
}  
}
```

```
// TestAccessModifier.java public  
class TestAccessModifier {
```

```
// Public instance variable: accessible everywhere  
public int publicVar = 10;
```

```
// Private instance variable: accessible only within this class  
private int privateVar = 20;
```

```
// Protected instance variable: accessible within this package and by subclasses  
protected int protectedVar = 30;
```

```
// Default (package-private) instance variable: accessible only within the same package  
int defaultVar = 40;
```

```
// Public method: accessible everywhere  
public void publicMethod() {  
    System.out.println("This is a public method.");  
}
```

```
// Private method: accessible only within this class  
private void privateMethod() {  
    System.out.println("This is a private method.");  
}
```

```
// Protected method: accessible within this package and by subclasses  
protected void protectedMethod() {  
    System.out.println("This is a protected method.");  
}
```

## LAB 8

```
// Default (package-private) method: accessible only within the same package
void defaultMethod() {
    System.out.println("This is a default method.");
}

// Method to demonstrate access to private members within the same class
public void accessPrivateMethod() {    privateMethod(); // Accessing
private method within the class
    System.out.println("Private Var from MyClass: " + privateVar);
}
}
```

### Task # 02

```
class Main {    public static void
main(String[] args) {    Dog dog = new
Dog("German", "Bully", "2");
    System.out.println(dog.breed);
    System.out.println(dog.name);
    System.out.println(dog.age);
}
}

class Animal{
protected String breed;
protected String name;
    protected String age;
}

class Dog extends Animal {
    Dog(String breed, String name, String age) {
        super.breed = breed;
super.name = name;    super.age
= age;
    } }
}
```

### Task # 03

```
public class Main {
    public static void main(String[] args) {
        Employee emp = new Employee("John Doe", 30, 50000);

        // Accessing the employee details through getter methods
        System.out.println("Name: " + emp.getName());
        System.out.println("Age: " + emp.getAge());
        System.out.println("Salary: $" + emp.getSalary());
    }
}
```

## LAB 8

```
// Modifying the employee details through setter methods
emp.setName("Jane Smith");    emp.setAge(28);
    emp.setSalary(55000);
```

```
emp.displayEmployeeDetails();
```

```
// Trying to set invalid values
emp.setAge(15); // Invalid age
emp.setSalary(-1000); // Invalid salary
}
}
```

```
class Employee {
    private String name;
    private int age;
    private double salary;

    public Employee(String name, int age, double salary) {
        this.name = name;
        this.age = age;
        this.salary = salary;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public int getAge() {
        return age;
    }

    public void setAge(int age) {
        this.age = age;
    }

    public double getSalary() {
        return salary;
    }
}
```

## LAB 8

```
    public void setSalary(double salary) {
this.salary = salary;
    }

    public void displayEmployeeDetails() {
System.out.println("Employee Name: " + name);
        System.out.println("Employee Age: " + age);
        System.out.println("Employee Salary: $" + salary);
    } }
}
```

### Task # 04

```
class main {
    public static void main(String[] args) {
        Bicycle myBicycle = new Bicycle("Giant", "Red", 21, 15.5);

        System.out.println("Initial Bicycle Details:");
myBicycle.displayBicycleDetails();
        System.out.println();

        // 2nd operation: Using setter methods to update the Bicycle properties
myBicycle.setBrand("Trek");    myBicycle.setColor("Blue");
myBicycle.setGearCount(18);    myBicycle.setSpeed(20.0);

        // Displaying updated bicycle details
System.out.println("Updated Bicycle Details:");
myBicycle.displayBicycleDetails();
        System.out.println();

        // 3rd operation: Using getter methods to access specific properties
System.out.println("Accessing specific properties using getters:");
        System.out.println("Brand: " + myBicycle.getBrand());
        System.out.println("Color: " + myBicycle.getColor());
        System.out.println("Gear Count: " + myBicycle.getGearCount());
        System.out.println("Speed: " + myBicycle.getSpeed() + " km/h");
        System.out.println();
    }
}

class Bicycle {    private
String brand;    private
String color;    private
int gearCount;    private
double speed;
```

## LAB 8

```
public Bicycle(String brand, String color, int gearCount, double speed) {
this.brand = brand;    this.color = color;    this.gearCount =
gearCount;    this.speed = speed;
}
public String getBrand() {
return brand;
}
public void setBrand(String brand) {
    this.brand = brand;
}
public String getColor() {
return color;
}
public void setColor(String color) {
    this.color = color;
}
public int getGearCount() {
return gearCount;
}
public void setGearCount(int gearCount) {
    this.gearCount = gearCount;
}
public double getSpeed() {
return speed;
}
public void setSpeed(double speed) {
    this.speed = speed;
}

public void displayBicycleDetails() {
System.out.println("Bicycle Details:");
    System.out.println("Brand: " + brand);
    System.out.println("Color: " + color);
    System.out.println("Gear Count: " + gearCount);    System.out.println("Speed: " + speed + " km/h");
} }
```

### Task # 05

```
class Main{
    public static void main(String[] args){
        Employee emp = new Employee(1, 10000, "John", "Doe");
        System.out.println(emp.getSalary());
        emp.raiseSalary();
    }
}
```

## LAB 8

```
        System.out.println(emp.getSalary());
    emp.getAnulSalary();
    emp.ToString();
}

class Employee{
    private int id , salary;    private
    String firstname, lastname;

    Employee(int id, int salary, String firstname, String lastname){
        this.id = id;        this.salary = salary;        this.firstname =
        firstname;        this.lastname = lastname;
    }
    int getSalary(){
        return salary;
    }
    void setSalary(int salary){
        if(salary < 0){
            System.out.println("Invalid salary");
        }else {
            this.salary = salary;
            System.out.println("Salary set to " + salary);
        }
    }
    int getAnulSalary(){
        return (this.salary*12);
    }
    int raiseSalary(){        int increment
    = (int) (0.1 * salary);        this.salary
    += increment;
    System.out.println("Salary raised to "
    + this.salary);
        return this.salary;
    }
    void ToString(){
        System.out.println("Employee Details");
        System.out.println("First Name: " + this.firstname);
        System.out.println("Last Name: " + this.lastname);
        System.out.println("ID: " + this.id + " Salary: " + this.salary);
    }
}
```